

Groundwork Round 2 - Agile Delivery Record

User stories, definitions of done, acceptance criteria, sprints, and timeline

Agile Delivery Record

This document shows how Round 2 of Groundwork was actually delivered: as a real, time-boxed sprint of work, with each feature defined as a user story, a clear finish line (definition of done), and testable proof it works (acceptance criteria) - not as a loose to-do list.

What this covers, and what it doesn't. This is project management for building the Groundwork platform itself. It does not apply to any single client diagnosis (like the Jane's Lending Co example used throughout demos) - that's a use of the finished product, not a development task with its own sprint.

Foundation: what already existed before this sprint

Not part of the 7-day sprint below, but worth stating clearly for the jury: this sprint built on top of a working platform, not a blank page.

Already in place	Source
Groundwork's multi-module architecture (Next.js, Vercel, module system)	Prior coursework, Project 3
Use Case Readiness diagnostic, live and working	Round 1
Platform Risk LangGraph agent skeleton (six-dimension scoring pipeline)	Project 3
Pinecone index with the authored framework corpus (book + ControlGap)	Project 3
Vercel Blob report storage	Round 1 (fixed after the live demo)

Timeline

Date	Day	Sprint
Fri 28 Aug 2026	Day 1	Sprint 1 - Compliance Foundation
Sat 29 Aug 2026	Day 2	Sprint 1
Sun 30 Aug 2026	Day 3	Sprint 1
Mon 31 Aug 2026	Day 4	Sprint 2 - Core Technical Builds
Tue 1 Sep 2026	Day 5	Sprint 2
Wed 2 Sep 2026	Day 6	Sprint 3 - Advanced Quality + Submission
Thu 3 Sep 2026	Day 7	Sprint 3

Three sprints, seven calendar days, worked continuously including the weekend.

Sprint 1 - Compliance Foundation

Fri 28 Aug - Sun 30 Aug

Goal: establish the legal and business reasoning that every later technical build would need to implement.

1.1: EU AI Act Risk Classification

As a BrainyFuture consultant, **I want** a documented EU AI Act risk classification for Jane's use case, **so that** I can tell her in writing whether her AI idea is safe to build under current regulation.

Definition of Done - `eu_ai_act_compliance.md` written and committed to the repo - Contains risk classification, mandatory requirements, conformity assessment, and a technical documentation outline

Acceptance Criteria - Given Jane's stated use case, when the four-question Annex III 5(b) boundary check is applied, then the document reaches a clear classification (Limited risk, as scoped) with reasoning shown - The document names the exact condition that would change the classification (a flag feeding underwriting)

1.2: GDPR Data Flow & Processing Register

As a BrainyFuture consultant, **I want** a documented GDPR data flow and processing register, **so that** I can show Jane exactly how her customers' data is handled and why it's lawful.

Definition of Done - `gdpr_documentation.md` written and committed - Contains a data flow map, processing register, a DPIA on the highest-risk step, data subject rights, and third-party transfers

Acceptance Criteria - Every processing activity in the register has a stated legal basis and retention period - The DPIA identifies at least three concrete risks and a mitigation for each

1.3: Business Case (ROI, Risk, Strategic Plan)

As a BrainyFuture consultant, **I want** a use case definition, ROI/risk assessment, and strategic deployment plan, **so that** I have a complete business case to present alongside the technical work.

Definition of Done - `use_case_definition.md`, `roi_risk_assessment.md`, `strategic_plan.md` written and committed

Acceptance Criteria - ROI is calculated for both 12 and 36 months, with a stated assumptions table - The risk matrix contains at least six risks across regulatory, technical, ethical, and operational categories - The strategic plan defines a greenlight condition between each phase

1.4: Round 2 Repo Skeleton

As a developer, **I want** the Round 2 repo folder structure in place, **so that** every subsequent deliverable has a defined home.

Definition of Done - `compliance/`, `poc/`, `mvp/`, `use_case_definition.md`, `roi_risk_assessment.md`, `strategic_plan.md` created and pushed to `capstone-round1`

Acceptance Criteria - `ls -R` on the repo shows the full structure matching the submission checklist - First commit and push succeed with no errors

| Sprint 2 - Core Technical Builds

Mon 31 Aug - Tue 1 Sep

Goal: turn Sprint 1's written reasoning into real, running, tested code inside the live Groundwork platform.

2.1: Jurisdiction-Aware Diagnostic

As a consultant using Groundwork, **I want** the diagnostic to know a client's jurisdiction, **so that** the EU AI Act check from Sprint 1 runs automatically on every diagnosis, not just on paper.

Definition of Done - Jurisdiction field added to the intake form (`lib/modules.ts`) - Prompt updated (`lib/prompts/readiness.ts`) with the regulatory boundary check - Jurisdiction shown on the report UI (`ReportView.tsx`)

Acceptance Criteria - Given a use case with jurisdiction "EU" that touches underwriting, when the diagnosis runs, then a disqualifier citing Annex III 5(b) is returned - Given no jurisdiction supplied, when the diagnosis runs, then the report explicitly states the gap rather than silently ignoring it - Verified live via curl and through the browser UI, not assumed from code review alone

2.2: Platform Risk Performance Fix

As a consultant using Groundwork, **I want** Platform Risk to return a result reliably, **so that** I can actually demo it without it timing out.

Definition of Done - `live_research_node` , `synthesis_node` , and `recommendation_node` parallelized with `ThreadPoolExecutor` - Deployed to Railway and confirmed active

Acceptance Criteria - Given a non-watchlisted vendor, when a report is requested, then it completes in under 5 minutes (Railway's hard platform limit) with real content in every section - Before/after timing recorded: 300+s failing to ~40-125s succeeding, worst case - Verified with a live `curl` request against the deployed Railway endpoint, response inspected for completeness

2.3: Incident-Grounded Evidence (RAG)

As a consultant using Groundwork, **I want** the Incident History dimension grounded in real documented AI failures, **so that** the risk score isn't based on a single generic web search.

Definition of Done - 1,641 AI Incident Database records ingested into a new Pinecone namespace (`incident-corpus`) - `retrieve_incident_matches` function and `incident_retrieval_node` built and wired into the LangGraph pipeline - `incident_matches` field added to the report schema

Acceptance Criteria - Given a real vendor query, when the graph runs, then real matched incidents appear on the final report with a calculated (not guessed) confidence score - A missing edge bug found during testing (synthesis → finalize_report) is identified and fixed, with the fix confirmed by inspecting the compiled graph object directly

2.4: Fix Router

As a consultant using Groundwork, **I want** a suggested next step after a "Not ready" verdict, **so that** I have something concrete to tell the client instead of just a no.

Definition of Done - `FixRouterSuggestion` type, `fix-router-store.ts` , `lib/prompts/fixRouter.ts` , and the `/api/reports/[id]/fix-router` route built - `FixRouterCard.tsx` UI component built and wired into both report views

Acceptance Criteria - Given a completed diagnosis's fix-first task, when Fix Router runs, then it returns 2-3 ranked suggestions, each with a stated confidence level and a real, checkable source - Suggestions persist after a page reload (saved to storage, not just held in browser memory) - A real strict-schema validation bug is found and fixed before this criterion passes

| Sprint 3 - Advanced Quality + Submission Prep

Wed 2 Sep - Thu 3 Sep

Goal: add an independent quality layer on top of Sprint 2's builds, then package everything for submission and the jury presentation.

3.1: LLM-as-Judge

As a consultant using Groundwork, **I want** an independent check on the diagnostic's own reasoning, **so that** I can trust a verdict before presenting it to a client.

Definition of Done - `JudgeReview` , `JudgeCheckedArea` types, `judge-store.ts` , `lib/prompts/judge.ts` , and the `/api/reports/[id]/judge` route built - `JudgeCard.tsx` built, showing a checked-areas table (always shows all 5 categories) and a findings table sorted by severity - Judge scope extended to also review Fix Router's suggestion quality

Acceptance Criteria - Given a completed report, when the judge runs, then it returns an agreement rating, a checked-areas table with no silent gaps, and any real findings ranked significant > moderate > minor - Verified live: the judge caught a real overclaiming issue in a Fix Router suggestion (a "strong evidence" label not actually supported by its cited source)

3.2: Diagnosis Timeout Reliability

As a consultant using Groundwork, **I want** the platform to handle slow AI responses gracefully, **so that** a real diagnosis never crashes with a confusing error, locally or in production.

Definition of Done - `instrumentation.ts` added to raise the local dev timeout - `maxDuration` and an `AbortController` timeout added to the production diagnose route, matched to the real Vercel plan ceiling

Acceptance Criteria - Given a diagnosis that takes several minutes, when run locally, then it completes without an `UND_ERR_HEADERS_TIMEOUT` crash - Given the same scenario in production, when it exceeds the safe window, then a clear `504` timeout message is returned instead of a silent platform kill

3.3: Submission Package

As a capstone presenter, I want a complete, professional submission package, so that the jury can evaluate the work without gaps.

Definition of Done - Presentation deck (12 slides), `mvp_documentation.md`, repo structure reference, and this agile delivery record all written and committed - A full re-check of both repos confirms no uncommitted work was left behind

Acceptance Criteria - `git status` on both repos returns a clean working tree - Every file referenced in the submission checklist exists in the repo with real content, verified by re-opening each one, not assumed from memory

Summary: 11 user stories, 3 sprints, 7 days

Sprint	Stories	Focus
Sprint 1	4	Legal & business reasoning
Sprint 2	4	Core technical builds
Sprint 3	3	Independent quality layer + submission

Every story above was verified against a real, running system before being marked done, not assumed correct from code alone. Several stories surfaced real bugs during testing (a missing graph edge, a strict-schema validation error, a 5-minute timeout), each one found through live testing and fixed before the story was closed.